codehive

Python Inheritance

Building Hierarchies and Reusing Logic

1. What is Inheritance?

Inheritance allows us to define a class that derives all methods and properties from another class. It facilitates **Code Reusability** and logical organization.

> **Parent Class (Base):** The class being inherited from.
>
> **Child Class (Derived):** The class that inherits from another.

2. Creating a Parent Class

The parent class is just a normal class. For our example, let's create a `Person` class that handles basic identity.

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)
```

3. Creating a Child Class

To inherit, pass the name of the parent class in parentheses when defining the child class.

```
class Student(Person):
  pass # Inherits everything!
```

Because `Student` inherits from `Person`, it automatically possesses the `firstname` property and the `printname()` method.

4. Overriding __init__

When you add an `__init__()` function to a child class, it **overrides** (replaces) the parent's constructor. This means the parent's attributes won't be set unless you call the parent manually.

If you define a new `__init__` in `Student`, the `firstname` from `Person` will not be initialized automatically.

5. Traditional Parent Call

You can explicitly call the parent's `__init__` method by using the Parent's class name. However, you must pass `self`.

```
class Student(Person):
  def __init__(self, fname, lname):
    Person.__init__(self, fname, lname)
```

This allows the parent to set up the shared data while the child can add its own specific logic below.

6. Using super()

Python provides the `super()` function. It automatically refers to the parent class without needing to name it.

```
class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)
```

Advantage: `super()` automatically handles `self` and makes your code more flexible if the parent class name changes.

7. Adding Specific Properties

Inheritance isn't just about copying; it's about **extension**. Here, we add a `graduationyear` to the Student.

```
class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

x = Student("Mike", "Olsen", 2019)
```

8. Adding Unique Methods

Child classes can have their own unique methods that are not available to the parent.

```
class Student(Person):
  def welcome(self):
    print(f"Welcome to the class of {self.graduationyear}")

# Person objects cannot call welcome()
# Student objects can call both printname() and welcome()
```

9. Polymorphism & Method Overriding

If a child class has a method with the **same name** as a parent method, the child's version wins. This is called overriding.

This allows you to change behavior specifically for a subtype (e.g., an `Admin` user might have a different `login()` logic than a `Guest`).

10. Practical Knowledge Check

Exercise

What is the correct keyword to use inside an empty class to avoid getting an error?

- ☐ empty
- ☐ inherit
- ☑ **pass**